total-ftp

Based off *Factoring "short-sleeve" RSA keys with polynomials* by Keegan Ryan (Trail of Bits)

article appendix: n_2

We know that n_2 is generated with the CompleteFTP RNG as follows:

```
public void genRandomBits(int bits) {
  // Calculate the number of limbs
  int numLimbs = bits / 32;
  // Allocate space for the RNG output
  byte[] array = new byte[numLimbs];
  // Call the system RNG
  rngProvider.GetNonZeroBytes(array);
  // Copy to the limbs of the big number
  Array.Copy(array, 0, bignumLimbs, 0, numLimbs);
  // Set the top bit to ensure proper bit length
  bignumLimbs[numLimbs - 1] |= 0x80000000;
  // Store the length
  dataLength = numLimbs;
}
```

As discussed in the (amazing) original article, the vulnerability lies in the fact that there is "a mismatch between the size of the limbs and the size of the RNG output".

1. If you compute $f_{n_2}(x)$ using $B = 2^{32}$, some of the coefficients are large. Why is that? Is it true that all of the coefficients of $f_p(x)$ and $f_q(x)$ are small?

Consider the coefficients of $f_p(x)$ and $f_q(x)$, i.e. the 32-bit limbs of p and q. From the definition of `genRandomBits()`, we know that each limb is actually a zero-extended 8-bit value, hence $x \in [1, 255] \cap \mathbb{Z}$ ***except the final limb***. The final limb (the most significant, assuming little-endian) has its top bit set. So the result of `genRandomBits(32 * 4)` would look like `[0xab, 0xcd, 0xef, 0x80000012]`.

Therefore it is not true that all of the coefficients of $f_p(x)$ and $f_q(x)$ are small: we know that the last coefficient of both will be the full 32 bits. Because this is the

Let N_i be the i-th 32-bit limb of n_2. By polynomial multiplication:

$$N_i = \sum_{j=0}^{i} p_j q_{i-j}$$

This implies that $\forall i \geq 31$, N_i must contain multiples of the final limb of p and q. So we can expect N_i to be more than 32-bit for $i \geq 31$. In practice due to carrying the values may be less than 32-bit, as we can observe from the limbs as shown below:

```
x=000029e5 bit_length=14
x=0000acdf bit_length=16
x=00010107 bit_length=17
x=000113f4 bit_length=17
x=000151be bit_length=17
x=0001a8a9 bit_length=17
x=0001cdf4 bit_length=17
x=00024384 bit_length=18
x=0002d5a6 bit_length=18
x=0002cbb3 bit_length=18
x=0002de11 bit_length=18
x=0003ada6 bit_length=18
x=0003fe6d bit_length=18
x=0004c564 bit_length=19
x=0004cfbe bit_length=19
x=000457dc bit_length=19
x=00040bf1 bit_length=19
x=000574dd bit_length=19
x=0005da92 bit_length=19
x=000566cc bit_length=19
x=0005ca4c bit_length=19
x=00060593 bit_length=19
x=00062231 bit_length=19
x=0006ec66 bit_length=19
x=0007c0ec bit_length=19
x=000724f1 bit_length=19
x=00072e59 bit_length=19
x=00080db8 bit_length=20
x=0007e843 bit_length=19
x=0009578f bit_length=20
x=0009a6d9 bit_length=20
x=00099a26 bit_length=20
```

```
x=800884c1 bit_length=32
x=8008d103 bit_length=32
x=80086398 bit_length=32
x=0008614d bit_length=20
x=80086095 bit_length=32
x=00073949 bit_length=19
x=8007f623 bit_length=32
x=80065147 bit_length=32
x=800765e6 bit_length=32
x=0006bb27 bit_length=19
x=8005ebe8 bit_length=32
x=80051cbd bit_length=32
x=80053b87 bit_length=32
x=80049249 bit_length=32
x=0004c270 bit_length=19
x=00058a21 bit_length=19
x=000418cb bit_length=19
x=00041021 bit_length=19
x=0003b283 bit_length=18
x=00039766 bit_length=18
x=80037c8c bit_length=32
x=0003cd44 bit_length=18
x=00034919 bit_length=18
x=80029cd5 bit_length=32
x=0002169f bit_length=18
x=0001be89 bit_length=17
x=8001e072 bit_length=32
x=8000b17b bit_length=32
x=00010ec5 bit_length=17
x=8000900e bit_length=32
x=000014ac bit_length=13
x=40000049 bit_length=31
```

The first column shows N_i for $i \in [0, 31]$, and the second column shows N_i for $i \in [32, 63]$. We can see that no values in the first column are greater than 20-bits, this is expected as they only involve 8-bit factors.

2. Is there a bit shift $p \ll i$ such that $f_{2^i p}(x)$ has small coefficients? This is the key trick needed to turn arbitrary short-sleeve values into polynomials with small coefficients.

Consider the last limb of p, p_0. Assume, WLOG, that

$$p_0 = \text{0x80000067} = \text{0b10000000000000000000000001100111}$$
$$\Rightarrow p_0 \ll 1 = \text{0x1000000ce} = \text{0b10000000000000000000000011001110}$$

Observe that $p_0 \ll 1$ overflows the 32-bit limb: in effect this turns the polynomial into a monic, with the second coefficient being $p_0 \ll 1 \bmod 2^{32} = \text{0xce}$. And because the rest of the coefficients of $f_p(x)$ are $x \in [\text{0x01}, \text{0xff}]$, $2x \in [\text{0x02}, \text{0x1fe}]$. So we know that $f_{2p}(x)$ has small coefficients, hence the bit shift we are looking for is $i = 1$.

3. If $f_{2^i p}(x)$ and $f_{2^j q}(x)$ have small coefficients, can you still compute $f_{2^i p}(x) * f_{2^j q}(x)$ from public information? Can you still recover p and q?

We know $i = 1$ (from q2). Because $f_{2p}(x)$ represents $2p$ as a polynomial, $f_{2p}(x) * f_{2q}(x)$ represents $2p \times 2q = 4n$ as a polynomial.

So we can factor f_{4n}, and halve the resulting values to recover p and q.

article appendix: n_1

We can utilise a similar method to solve for n_1. p and q are generated as follows:

```
import random
import Crypto.Util.number
def gen_n1_prime():
    count = 1024 // 128
    w = 128
    b = 32
    p = 0
    for i in range(count):
        pi = random.randrange(2**b)
        p += 2**(w*i) * pi
    p <<= 96
    while not Crypto.Util.number.isPrime(p):
        p += 1
    return p
```

We can observe a few differences to the CompleteFTP prime RNG:

- Limb size is 128-bit vs 32-bit
- We generate 32-bits instead of 8-bits
- We do a left shift by 96 bits at the end
  - this is interesting: does this make our primes 1120-bit instead?
  - ***no:*** `range(count)` goes from $[0, 7]$, so p_7 is at bits $[128 \times 7, 128 \times 7 + 32) = [869, 928)$. Hence the `p <<= 96` causes the top end of p_7 to be at bits $[992, 1024)$, making this a 1024-bit prime

Let's revisit questions 1-3 to try and attack this prime.

1. If you compute $f_{n_1}(x)$ using $B = 2^{128}$, some of the coefficients are large. Why is that? Is it true that all of the coefficients of $f_p(x)$ and $f_q(x)$ are small?

We have these 128-bit limbs from n_1:

```
x=89ac77db000000000000000000036a77 bit_length=128
x=f0f7d3bfdce03d6000000000000026b bit_length=124
x=c5e55654e9b14cba000000000000040f bit_length=128
x=c495b3690a69b66c00000000000000d9 bit_length=128
x=a82a59342aadff570000000000000295 bit_length=128
x=1291f9aaa7e9a7d60000000000000337 bit_length=125
x=d762d68bcbe8cc3a00000000000000d3 bit_length=128
x=455c59eec3a0654500000000000003f8 bit_length=127
x=de525d2b1011ceae0000000000000424 bit_length=128
x=6ee98c677dfdf190000000000000002 bit_length=123
x=e8e3c0efdd8054ba0000000000000003 bit_length=128
x=2f743377005a840d0000000000000001 bit_length=126
x=1036d671407a0660000000000000002 bit_length=121
x=2c16eeaeab96ddc80000000000000002 bit_length=126
x=4d2ee8284c7a03c00000000000000001 bit_length=127
x=c889f7ef523b08e40000000000000001 bit_length=128
```

We might expect the 32-bit limbs of p and q to produce 64-bit limbs of n_1, but because p and q have been left shifted, the 32-bit have ended up in the MSBs of each limb. So in effect we get two 128-bit numbers being multiplied, with carrries as seen in the LSBs of the n_1 limbs.

2. Is there a bit shift $p \ll i$ such that $f_{2^i p}(x)$ has small coefficients? This is the key trick needed to turn arbitrary short-sleeve values into polynomials with small coefficients.

Let's generate a prime ourselves and take a look at its limbs:

```
n1_prime.bit_length()=1024
x=b8877081000000000000000000000a7 bit_length=128
```

```
x=24a8d64d00000000000000000000000 bit_length=126
x=7b9a178400000000000000000000000 bit_length=127
x=ef4746e600000000000000000000000 bit_length=128
x=3626004800000000000000000000000 bit_length=126
x=2e86aebb00000000000000000000000 bit_length=126
x=251d791300000000000000000000000 bit_length=126
x=dab0150600000000000000000000000 bit_length=128
```

As expected, the bottom 96 bits are empty, with exception of the least-significant limb having its low bits set to make it a prime.

But what if we take 32-bit limbs?

```
x=000000a7 bit_length=8
x=00000000 bit_length=0
x=00000000 bit_length=0
x=b8877081 bit_length=32
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=24a8d64d bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=7b9a1784 bit_length=31
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=ef4746e6 bit_length=32
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=36260048 bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=2e86aebb bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=251d7913 bit_length=30
x=00000000 bit_length=0
x=00000000 bit_length=0
x=00000000 bit_length=0
x=dab01506 bit_length=32
```

Perfect! Every fourth limb is 32-bit, and we have an 8-bit (in this case) least-significant limb.